

Propeller 2 • Application Note P2AN006

Sizing Cog & Task Stacks

Give every cog and task exactly the stack it needs — and catch an overflow before it corrupts memory

October 2026

Version 1.0.2

What You'll Build

A repeatable way to size the stack buffer every cog and task needs — and a drop-in utility that turns a silent stack overflow into a clear, halted error message.

One shared idea — fill a stack with a known pattern and watch a sentinel just past its end — then a small catalog of recipes you choose among by need:

- **R1 — instrument a new-cog stack** — guard a `cogspin` worker against overflow
- **R2 — find the high-water mark** — measure real usage, then right-size
- **R3 — pinpoint the culprit** — localize which routine overran the stack
- **R4 — size a task stack** — the same technique on an intra-cog `taskspin` task

Applies to: P2 (Propeller 2) silicon • Spin2 / PNut (v47+ for the task recipe) • P2 Edge or P2 Eval board; every recipe reports over DEBUG.

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Abstract

Every cog you start with `cogspin`, and every cooperative task you start with `TASKSPIN`, needs a stack — a block of hub memory you hand it for its calls, parameters, and locals. Hand it too little and the P2 gives you no warning: the stack silently overruns its buffer and corrupts whatever memory sits after it, producing bugs that seem to come from nowhere. This note shows how to size those stacks with confidence — how to measure what a cog or task actually uses, how to catch an overflow the instant it happens, and how to pin down which routine caused it. The workhorse is a small, drop-in utility that fills the stack with a known pattern and watches a sentinel just past its end.

What you'll build: four short programs that instrument a stack — one that guards a cog against overflow, one that measures real usage so you can right-size, one that pinpoints the overrunning routine, and one that does the same for a cooperative task's stack.

Prerequisites & Hardware

You should already be comfortable with:

- Reading and writing Spin2 — methods, `DAT`/`VAR` blocks, and `DEBUG()`.
- Launching a method in a new cog with `cogspin` (and, for R4, launching a cooperative task with `TASKSPIN` — see the companion note **P2AN005, Cooperative Multitasking with Spin2 TASK Methods**, a companion P2 Knowledge Base publication).
- The idea that a call stack holds a method's parameters, return values, locals, and its chain of nested calls.

Version gate. R1–R3 are ordinary Spin2 and need no directive. R4 uses `TASKSPIN`, so it opens with `{Spin2_v47}` — required for the TASK methods. The stack-check utility itself uses only `DEBUG()`.

Hardware:

- A P2 Edge or P2 Eval board. Every recipe reports over DEBUG, so you can watch each work with no extra parts.
- Build with DEBUG enabled — `pnut-ts -d` — because the utility reports through `DEBUG()`. Without `-d` the compiler skips the debug output and you'll see nothing.

The Idea

A stack is a buffer you own. When you write

```
DAT
  workerStack  LONG 0[64]
```

and launch a cog or task with `@workerStack`, you are promising that the code will never need more than 64 longs of stack. If it does — a deeper CALL chain than you expected, a method with more

locals than you counted — the stack keeps growing **right past the end of your buffer**, writing into whatever variable, buffer, or code happens to live next in hub memory. Nothing traps it. The symptom is not a clean error; it is a distant variable mysteriously changing, or a crash with no obvious cause.

There are two places you hand over a stack, and they behave the same way:

- A **new-cog stack** — the buffer you pass to `cogspin` (or `COGINIT`). Each cog gets its own independent CALL stack.
- An **intra-cog task stack** — the buffer you pass to `TASKSPIN`. Each cooperative task in a cog has its own stack, separate from its siblings'.

Either way, the P2 does not check the boundary for you, so you check it yourself. The trick is simple: before launch, **fill the whole stack buffer with a recognizable pattern**, and place a distinctive **sentinel** value in the long immediately *after* the buffer. The stack grows into the pattern from the bottom; the first thing an overflow overwrites is that sentinel. Read the sentinel back — if it still holds its value, the stack never crossed the line; if it changed, you overflowed. And by finding the deepest long that no longer holds the pattern, you learn exactly how deep the stack actually got — the **high-water mark** — which is what lets you size it right.

That whole idea is packaged in a small MIT-licensed utility, **`isp_stack_check`** (by Stephen M. Moraco), which this note uses as its instrument. You drop it into your project, prepare the stack before launch, and check it while the code runs.

You may well have built something like this yourself. Stack instrumentation is a common rite of passage, and plenty of P2 developers already carry their own version. `isp_stack_check` appears here as a complete, concrete example of the technique — not a mandate. Use it if you'd like a ready-made instrument; keep your own if you have one. Either way, the concepts this note works through — a fill pattern, a sentinel just past the buffer, a high-water mark, and a cheap runtime guard — make a checklist you can hold your own stack-check code against, to be sure it covers everything it should.

How It Works — the sentinel technique

`isp_stack_check` uses two magic constants: a fill pattern and a sentinel.

Constant	Value	Role
`NOT_WRITTEN_MARK`	`$a5a50df0`	written into **every** long of the stack buffer before launch
`DO_NOT_WRITE_MARK`	`$addee5e5`	the **sentinel**, placed in the long **just past** the buffer

Its stack methods all take the stack pointer and the buffer size **in longs**:

Method	What it does
`prepStackForCheck(@stack, longs)`	Fill the buffer with `NOT_WRITTEN_MARK` and drop the sentinel just past it. **Call before launch.**
`checkStack(@stack, longs)`	If the sentinel was overwritten, print an overflow message and halt for inspection. **Call while running.**
`isStackGuardIntact(@stack, longs)`	Return TRUE while the sentinel holds. Prints nothing and halts nothing, so a test can score an overflow and carry on.
`getStackDepth(@stack, longs)`	Return the high-water mark: scan down from the top of the buffer to the deepest long that no longer holds the fill pattern.
`reportStackUse(@stack, longs)`	Call `getStackDepth` and print `STACK used N of M`.

The utility holds no state of its own: every method works on the buffer you pass it. So any number of cogs may check their own stacks at the same time. It also carries a second, independent detector for buffers other than stacks: `initByteGuard`, `isByteGuardIntact` and `firstCorruptOffset` fill a zone of guard bytes after a buffer and report the first byte that changed.

There is one placement rule that makes the whole thing work: **the sentinel long must physically abut the buffer, with no gap.** In Spin2 you get that by declaring the sentinel immediately after the stack in the same `DAT` block:

```
DAT
  workerStack   LONG 0[STACK_LONGS]
  endStackMark  LONG stk.DO_NOT_WRITE_MARK  ' sentinel MUST abut buffer
```

`prepStackForCheck` will write the sentinel into that long for you; declaring it here guarantees the long exists and sits exactly where the check expects it.

How to size a stack

The measurement gives you a method, not a guess:

1. **Start generous.** A cog running moderate Spin2 code typically wants **64–128 longs**; the absolute floor is around 32. Begin at **128** so you can't overflow while measuring. (Source: the P2 Knowledge Base `cogspin` guidance.)
2. **Exercise the deepest path.** Run the code through its worst case — the deepest CALL chain, the most-nested routine — because the high-water mark only reflects paths that actually ran.
3. **Read the high-water mark** with `reportStackUse`. It prints `STACK used N of 128`.
4. **Right-size with headroom.** Set the buffer to about **1.5 × N** and keep a `checkStack` in the loop as a permanent guard.

★ **Tip — let a regression test drive the worst case.** Step 2 is only as good as the paths you remember to exercise. Rather than hand-craft a worst-case run, point your normal **regression test** at the cog or task — one that exercises every feature, mixing the deep and the short paths the way real use does — and read the stack *after* it finishes. Because it covers the whole feature set, it drives the genuine deepest path for you, so the high-water mark it leaves behind is a far stronger, more trustworthy measurement than a hand-picked spot check.

What consumes the stack, in order, is each method's **parameters, then its return values, then its locals** — packed as they are declared — plus the depth of nested calls. (Source: the Spin2 language documentation.) A note on scope: this is the **hub** CALL stack that Spin2 methods use. It is a different resource from the P2's small **hardware** CALL stack (the 8-level `PUSH`/`POP`/`CALL` stack that PASM uses); sizing the hub stack here has nothing to do with that hardware stack.

Choosing a Recipe

Pick by what you're doing:

If you want to...	Use	Method(s)
guard a cog against silent overflow	**R1** — instrument a cog stack	`prepStackForCheck`, `checkStack`
measure real usage and right-size	**R2** — high-water mark	`reportStackUse` / `getStackDepth`
find *which* routine overran the stack	**R3** — pinpoint the culprit	granular `checkStack`
size a cooperative task's stack	**R4** — size a task stack	the same, on a `TASKSPIN` buffer

Each recipe is a complete, runnable program that needs `isp_stack_check.spin2` beside it, and each ends with a ♦ **Verify**.

Recipe R1 — Instrument a New-Cog Stack

Use it when you launch a worker in a new cog and want a silent overflow turned into a loud, halted error instead of distant corruption.

```
CON
  _clkfreq = 200_000_000
  STACK_LONGS = 64                          ' the worker cog's stack (longs)

OBJ
  stk : "isp_stack_check"

DAT
  workerStack   LONG 0[STACK_LONGS]
  endStackMark  LONG stk.DO_NOT_WRITE_MARK  ' sentinel MUST abut buffer

PUB main()
  ' Phase 1+2: fill buffer with pattern, drop sentinel, THEN launch.
  stk.prepStackForCheck(@workerStack, STACK_LONGS)
  cogspin(NEWCOG, worker(), @workerStack)

  ' Phase 3: watch the sentinel. checkStack halts the program
  ' if the worker overran STACK_LONGS.
  repeat
    stk.checkStack(@workerStack, STACK_LONGS)
    waitms(500)

PRI worker() | scratch
  repeat
    scratch := doUnitOfWork()               ' the cog's real work goes here
    waitms(10)
```

continues on next page →

↪ continued from previous page

```
PRI doUnitOfWork() : result
  result := GETCT()


                                                    instrument-cog-stack.spin2
```

How this works. The three phases are the whole pattern. **Prepare** (`prepStackForCheck`) fills `workerStack` with the pattern and writes the sentinel into `endStackMark`, the long that abuts it — all *before* the worker starts, so the baseline is clean. **Launch** with `cogspin`, handing the worker that same buffer as its stack. **Check** in the loop: each `checkStack` reads the sentinel; as long as the worker stays within 64 longs, the sentinel is intact and nothing happens. The moment the worker's stack grows past 64 longs, it overwrites the sentinel, and the next `checkStack` prints `STACK Overflow!` and halts so you can see it.

★ **Tip — the guard costs almost nothing.** `checkStack` reads one long and compares it. Leaving it in a slow supervisory loop (here, twice a second) is cheap insurance you can ship.

♦ **Verify:** With `STACK_LONGS = 64` and this light worker, nothing prints — the sentinel is never touched, which *is* the pass. To see the instrument fire, shrink `STACK_LONGS` to a value smaller than the worker really needs (try deepening `doUnitOfWork` with a few nested calls first); the next `checkStack` prints `^^^ STACK Overflow! Depth greater than N longs` and halts. If you see the overflow message on the very first check with a generous stack, confirm `endStackMark` immediately follows `workerStack` in the *same* `DAT` block — a gap between them means the sentinel isn't where the check looks.

Recipe R2 — Find the High-Water Mark, then Right-Size

Use it when you want to stop guessing and set the stack to what the code actually uses.

```
CON
  _clkfreq = 200_000_000
  ' START GENEROUS; shrink after you read the high-water mark
  STACK_LONGS = 128

OBJ
  stk : "isp_stack_check"

DAT
  workerStack   LONG 0[STACK_LONGS]
  endStackMark  LONG stk.DO_NOT_WRITE_MARK

PUB main()
  stk.prepStackForCheck(@workerStack, STACK_LONGS)
  cogspin(NEWCOG, worker(), @workerStack)

  ' Give the worker time to run its worst-case path once, THEN measure.
  waitms(1000)
  ' prints: "^^^ STACK used N of 128"
```

continues on next page →

↪ continued from previous page

```
  stk.reportStackUse(@workerStack, STACK_LONGS)

  ' From here: set STACK_LONGS to ~1.5 x N; re-run, keep the guard.
  repeat
    stk.checkStack(@workerStack, STACK_LONGS)
    waitms(500)

PRI worker() | r
  ' High-water mark reflects only paths that ran; exercise the deepest.
  r := deepChain(6)
  repeat
    waitms(10)

PRI deepChain(n) : r | localWork
  ' Recursion drives stack depth (each call frames its parameter + local).
  localWork := n * 3
  if n <= 0
    return localWork
  r := deepChain(n - 1) + localWork
```

high-water-mark.spin2

How this works. The buffer starts at a deliberately generous 128 longs so the measurement itself can't overflow. The worker runs its deepest path once — here `deepChain(6)`, whose recursion frames a parameter and a local at each level, so it drives real stack depth. After giving it time to run, `reportStackUse` calls `getStackDepth`, which scans down from the top of the buffer to the deepest long that no longer holds the fill pattern. That position is the high-water mark, and it prints `STACK used N of 128`. Now you know the real number: set `STACK_LONGS` to about **1.5 × N**, re-run, and keep the `checkStack` guard for the paths your test didn't exercise.

▲ **Pitfall — measure the worst case, not a quiet moment.** `getStackDepth` only sees what actually ran. If your test never triggers the deepest error handler or the most-nested branch, the high-water mark undercounts and you'll size the stack too small. Drive the code through its worst case *before* you read the mark.

♦ **Verify:** DEBUG prints one `^^^ STACK used N of 128` line. Increase `deepChain(6)` to `deepChain(12)` and the reported N climbs — direct confirmation that deeper CALL chains use more stack. That relationship is the recipe: the number you read is what you size to (plus headroom).

Recipe R3 — Pinpoint the Overflowing Routine

Use it when you *know* something overflows but not *what*. One check at the top of the loop tells you the stack blew; a check after each step tells you which step blew it.

```
CON
  _clkfreq = 200_000_000
  STACK_LONGS = 64
```

continues on next page →

↪ continued from previous page

```
OBJ
  stk : "isp_stack_check"

DAT
  workerStack   LONG 0[STACK_LONGS]
  endStackMark  LONG stk.DO_NOT_WRITE_MARK

PUB main()
  stk.prepStackForCheck(@workerStack, STACK_LONGS)
  cogspin(NEWCOG, worker(), @workerStack)
  repeat
    waitms(1000)

PRI worker()
  ' Granular checks: whichever halts names the routine that overran it.
  initSensors()
  stk.checkStack(@workerStack, STACK_LONGS)     ' clean after sensors?
  initComms()
  stk.checkStack(@workerStack, STACK_LONGS)     ' HERE: initComms overran
  initControl()
  stk.checkStack(@workerStack, STACK_LONGS)     ' clean after control?
  repeat
    waitms(10)

PRI initSensors() | a
  a := GETCT()

PRI initComms() | b
  b := GETCT()

PRI initControl() | c
  c := GETCT()
```

pinpoint-overflow.spin2

How this works. After each init routine returns, `checkStack` reads the sentinel. Because the check runs *between* routines, the first one that reports an overflow tells you the routine that just returned is the one whose CALL chain crossed the line — the halt lands right after the culprit. Note that the worker cog checks *its own* stack here: the check runs inside `worker`.

★ **Tip — bisect a deep chain.** If a single routine's subtree overflows, drop `checkStack` calls *inside* it, around its sub-steps, and repeat. Each halt narrows the search until you're standing on the exact call that costs too much.

♦ **Verify:** With a 64-long stack and these trivial inits, nothing halts. To watch it localize a fault, give one routine a genuinely deep sub-chain (a few levels of nested calls with locals) and shrink `STACK_LONGS`; the `checkStack` immediately after *that* routine is the one that fires. The routine named is the one to shrink — or the reason to grow the stack.

Recipe R4 — Size a Task Stack

Use it when you're sizing the stack of a cooperative task (from **P2AN005**), not a new cog. The instrument doesn't care which — it only watches a hub buffer — so the exact same technique applies.

```
{Spin2_v47}
CON
  _clkfreq = 200_000_000
  TASK_STACK_LONGS = 64  ' the cooperative task's stack (longs)

OBJ
  stk : "isp_stack_check"

DAT
  taskStack     LONG 0[TASK_STACK_LONGS]
  endStackMark  LONG stk.DO_NOT_WRITE_MARK

PUB main() | reports
  stk.prepStackForCheck(@taskStack, TASK_STACK_LONGS)
  taskspin(NEWTASK, worker(), @taskStack)  ' cooperative task, not a new cog

  reports := 0
  repeat
    stk.checkStack(@taskStack, TASK_STACK_LONGS)  ' guard: halts on overflow
    if reports++ // 20 == 0
      ' periodic high-water read
      stk.reportStackUse(@taskStack, TASK_STACK_LONGS)
    waitms(50)
    tasknext()

PRI worker() | r
  repeat
    r := taskWork(4)
    tasknext()

PRI taskWork(n) : r | localWork
  localWork := n + 1
  if n <= 0
    return localWork
  r := taskWork(n - 1) + localWork
```

size-task-stack.spin2

How this works. This is R1 and R2 combined, aimed at a `TASKSPIN` buffer instead of a `cogspin` one. `taskStack` is prepared exactly as before, the task is launched with `TASKSPIN(NEWTASK, ...)`, and the supervisor task guards the buffer with `checkStack` every pass while printing the high-water mark every twentieth pass. The point of the recipe is the equivalence: **sizing a task stack is the same job as sizing a cog stack.** A task tends to need *less* than a full cog because it does one focused job, but the method — start generous, measure, right-size — is identical.

♦ **Verify:** DEBUG prints a `^^^ STACK used N of 64` line every twentieth supervisor pass, with the guard silent as long as `worker` stays within budget. Deepen `taskWork(4)` to `taskWork(20)` and watch N rise; shrink `TASK_STACK_LONGS` below what the task needs and the guard's `checkStack` fires the overflow halt — proving the technique works identically on a task stack.

Adapt It / Going Further

A few habits carry everything here into real projects:

- **Ship the guard, not just the measurement.** Sizing tells you what a stack needs *for the paths you tested*. A `checkStack` left in a slow supervisory loop costs one long-compare and turns any future overflow — a new deep branch, a bigger input — into a clear halt instead of silent corruption. Keep it in.
- **Can't afford the guard in shipping code? Compile it out.** The guard is cheap, so ship it if you can. But when a production build truly can't spare the cycles or the longs, wrap the `prepStackForCheck` and `checkStack` calls in Spin2's conditional-compilation directives so they leave the shipping binary entirely: gate them on `#ifdef __DEBUG__` — a symbol the compiler defines automatically whenever you build with `pnut-ts -d` — and the instrument is fully present while you develop and test, then compiles to nothing in the non-DEBUG shipping build. For control independent of DEBUG, define your own switch instead (`#ifdef STACKGUARD`, enabled with `-D STACKGUARD` on the command line while you need it). *(Spin2 pre-processor, v48+.)*
- **Size every launch.** Every `cogspin` and every `TASKSPIN` hands over a stack; each is a place this note applies. When you launch several cooperative tasks in one cog (P2AN005), size each task's stack the same way — a task that does one focused job usually needs far less than a full cog, and measuring lets you reclaim the difference.
- **In a misbehaving multi-cog system, rule out stack overflow first.** Silent overflow is one of the sneakiest faults in a complex design: the corruption surfaces far from its cause, as symptoms that look like unrelated bugs. Before chasing them, instrument and right-size *every* cog and task stack in the system. Clearing all the stack-depth issues first collapses the overflow-induced noise and leaves a much smaller symptom set — the failures that reflect actual logic bugs worth fixing.
- **Check several cogs at once if you need to.** The utility keeps no state between calls, so each cog can guard and measure its own stack concurrently. Give each stack its own sentinel long.
- **Score an overflow without halting.** In a test that should record the failure and keep going, use `isStackGuardIntact` instead of `checkStack`.

Pitfalls & Notes

▲ **Pitfall — the sentinel long must abut the buffer.** Declare `endStackMark` immediately after the stack in the *same* `DAT` block. A gap between them puts the sentinel somewhere the overflow won't reach first (or the check reads the wrong long), and the whole technique quietly stops working.

▲ **Pitfall — the high-water mark only counts paths that ran.** `getStackDepth` reports the deepest the stack *actually* got during your test. If the worst case never executed, you'll size too small. Drive the code through its deepest path before you trust the number.

▲ **Pitfall — `checkStack` proves the boundary was crossed, not that all is well.** It verifies exactly the one long past the buffer. Because a stack grows contiguously, an overflow always clobbers that sentinel first, so detection is sound — but read the result as "the boundary held / was crossed," not as a full memory-integrity check.

▲ **Pitfall — a reading equal to the buffer size is a floor, not a measurement.** If `getStackDepth` returns the full buffer size, the topmost long has been used, and the scan cannot tell how much further the cog would have gone. Two different causes leave the same memory behind: the stack overflowed, or `prepStackForCheck` never ran, so the buffer holds zeros instead of the pattern. Check `isStackGuardIntact` to tell an overflow apart, and confirm that the prepare step ran.

▲ **Pitfall — never let two guards share an address.** If you also put a byte guard after a buffer (`initByteGuard`), keep a spare long between it and any stack sentinel. The two detectors want different values in the same place, only the last writer wins, and one of them then reports a failure that never happened.

★ **Tip — start at 128, settle at measured × 1.5.** 128 longs is a safe starting point for most Spin2 code; the floor is around 32. Measure, then set the buffer to roughly 1.5× the high-water mark so a slightly deeper future path still fits under the guard.

■ **Hardware — this is the hub stack, not the hardware stack.** The buffer you size here is the Spin2 method call stack in hub RAM. It is unrelated to the P2's 8-level hardware CALL stack used by PASM `CALL`/`RET` — sizing one has no bearing on the other.

Conclusion

The P2 will not tell you when a stack is too small; it just corrupts memory and leaves you chasing a ghost. The fix is to measure instead of guess. Fill the buffer with a pattern, watch a sentinel just past its end, and you get two things at once: a high-water mark that tells you how big the stack really needs to be, and a cheap runtime guard that turns any overflow into a clear, halted error. The technique is identical for a new cog (`cogspin`) and a cooperative task (`TASKSPIN`) — the instrument only watches a hub buffer. Start generous, exercise the worst case, right-size with headroom, and leave the guard in. Do that for every launch and stack overflow stops being a mystery. When you launch cooperative tasks to size, the companion note **P2AN005** shows how they run.

Resources

- **Example library** — every recipe plus the instrument: `isp_stack_check.spin2` (the utility), `instrument-cog-stack.spin2` (R1), `high-water-mark.spin2` (R2), `pinpoint-overflow.spin2` (R3), `size-task-stack.spin2` (R4). Published as `P2AN006-src.zip` beside this note.
- **`isp_stack_check`** by Stephen M. Moraco (Iron Sheep Productions, LLC) — the MIT-licensed stack-check utility this note is built around.

- **P2AN005 — Cooperative Multitasking with Spin2 TASK Methods** (a companion P2 Knowledge Base publication) — how the tasks whose stacks R4 sizes are launched and run.

References

1. *Spin2 Language Documentation* (Chip Gracey, Parallax Inc.) — `cogspin` / `COGINIT` stack requirements, `TASKSPIN` task stacks, and the parameters-then-results-then-locals stack layout.
2. P2 Knowledge Base `cogspin` guidance — typical stack sizing (32-long floor, 64–128 typical) and the note that stack overflow is not detected in hardware.
3. `isp_stack_check.spin2`, Stephen M. Moraco (Iron Sheep Productions, LLC), MIT License — the sentinel-fill stack-overflow utility.

Revision History

- **v1.0.2** (October 2026) — the bundled `isp_stack_check` utility is its current version. The depth reading finds the high-water mark by scanning down from the top of the buffer. The utility holds no state between calls, so several cogs can check their own stacks at once. It adds an overflow check that does not halt, and a byte-guard detector for other buffers. New pitfalls: a full-size reading is a floor, and two guards must never share an address.
- **v1.0.1** (August 2026) — license restored to CC BY-SA 4.0: share and adapt, including commercially, with attribution and under the same terms. The Trademarks note now states the license grants permissions under copyright only. No technical content changed.
- **v1.0.0** (July 2026) — initial release for community review. A techniques-catalog for sizing the stack buffers that `cogspin` (new-cog) and `TASKSPIN` (intra-cog task) require on the P2 — where an undersized stack silently overwrites hub memory with no hardware trap. One shared idea (fill the stack with a known pattern and watch a sentinel just past its end) applied through four runnable recipes: instrument a new-cog stack against overflow, find the high-water mark and right-size, pinpoint which routine overran the stack, and size a cooperative task's stack (the companion to P2AN005). Built around the MIT-licensed `isp_stack_check` utility; every worked program compiles clean under `pnut-ts -d`.

Copyright & License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0) — you may share and adapt it, including commercially, with attribution and under the same terms. To view the full license, visit `https://creativecommons.org/licenses/by-sa/4.0/`.

Parallax, Propeller, and Spin are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed

copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC or Parallax Inc.

The bundled `isp_stack_check.spin2` utility is © 2024 Iron Sheep Productions, LLC and is distributed under the MIT License; its license text is retained in the file.

Acknowledgments

Parallax Inc. and Chip Gracey — for the Propeller 2 silicon and the Spin2 language. Stephen M. Moraco (Iron Sheep Productions, LLC) — for the `isp_stack_check` utility that anchors this note.